

Chained Multiple Imputation Workflows with `mimar`

Overview

Missing data are ubiquitous in applied statistics. A large proportion of real data sets contain at least some incomplete cases, and the way missingness is handled can substantially affect point estimates, standard errors, and downstream inference. Complete-case analysis discards potentially informative observations and, unless data are missing completely at random, introduces selection bias. Single imputation fills missing values once but ignores imputation uncertainty, leading to standard errors that are too narrow.

Multiple imputation addresses both problems. By generating m plausible completions of the data and combining results using Rubin's pooling rules (Rubin 1987), it propagates uncertainty about what the missing values might have been into the final estimates. The chained-equation formulation, sometimes called fully conditional specification, makes multiple imputation tractable for mixed-type data by modeling each incomplete variable conditionally on all others, cycling through variables until the chained summaries appear stable (Buuren and Groothuis-Oudshoorn 2011).

`mimar` is designed for analyses where missing-data handling, artificial missingness benchmarks, imputation diagnostics, and downstream pooling should live in one compact grammar. Its contribution is not to replace the mature special-purpose imputation ecosystem, but to make a complete workflow concise: describe the missingness, create benchmark amputations when needed, impute with native or learner-backed update rules, inspect diagnostics, evaluate recovered cells when truth is available, and pool post-fit quantities.

`mimar` organizes this workflow into a small set of composable functions:

```
describe()      # characterise missingness structure
ampute()        # introduce controlled artificial missingness
imputer_registry() # inspect available imputation methods
impute()        # run the chained multiple imputation
complete()      # extract a single completed dataset
evaluate()      # score imputation quality against known truth
pool()          # apply Rubin's pooling rules
plot()          # visualise any of the above objects
```

The central design decision is that `mimar` owns the outer chained-equation loop and multiple-imputation bookkeeping. Each imputer is simply a variable-level update rule slotted into that loop. This separation means that native parametric methods, donor-based methods, and machine-learning-backed methods are called identically and obey the same convergence and type-restoration logic:

```
impute(data, imputer = "pmm", m = 5, maxit = 5)
impute(data, imputer = "rf", m = 5, maxit = 5)
impute(data, imputer = "xgboost", m = 5, maxit = 5)
```

No external modelling framework is required. Learner-backed imputers call their original packages directly, and those backend packages are installed with `mimar` so all registered imputers are available without extra installation steps.

Succinct Use

For everyday work, `impute()` is the only function you need. The input data can contain `NA`, and the completed outputs returned by `complete()` do not.

```

succinct_dat <- data.frame(
  x = c(1, NA, 3, 4, NA),
  z = factor(c("a", "b", NA, "a", "b"))
)

i <- mimar::impute(succinct_dat, imputer = "knn", m = 2, maxit = 2, seed = 1)
mimar::complete(i, 1)
#> # A tibble: 5 x 2
#>       x z
#>   <dbl> <fct>
#> 1     1 a
#> 2     4 b
#> 3     3 b
#> 4     4 a
#> 5     1 b
mimar::complete(i, "all")
#> [[1]]
#> # A tibble: 5 x 2
#>       x z
#>   <dbl> <fct>
#> 1     1 a
#> 2     4 b
#> 3     3 b
#> 4     4 a
#> 5     1 b
#>
#> [[2]]
#> # A tibble: 5 x 2
#>       x z
#>   <dbl> <fct>
#> 1     1 a
#> 2     1 b
#> 3     3 a
#> 4     4 a
#> 5     4 b

```

The workflow can be summarized as:

Stage	Function	Main object
Missingness description	<code>describe()</code>	<code>mimar_missing</code>
Artificial missingness	<code>ampute()</code>	<code>mimar_amputation</code>
Imputation	<code>impute()</code>	<code>mimar_imputation</code>
Completed data extraction	<code>complete()</code>	data frame or list
Quality assessment	<code>evaluate()</code>	<code>mimar_evaluation</code>
Post-fit pooling	<code>pool()</code>	<code>mimar_pool</code>
Diagnostics	<code>plot()</code>	<code>ggplot</code>

```

library(mimar)

set.seed(1)
dat <- data.frame(
  age = rnorm(120, 50, 10),

```

```

bmi   = rnorm(120, 25, 4),
sex   = factor(sample(c("F", "M"),      120, TRUE)),
group = factor(sample(c("A", "B", "C"), 120, TRUE)),
smoker = sample(c(TRUE, FALSE),        120, TRUE)
)

head(dat)
#>      age      bmi sex group smoker
#> 1 43.73546 22.97617 M    C  FALSE
#> 2 51.83643 30.37216 F    A  FALSE
#> 3 41.64371 24.14168 F    B   TRUE
#> 4 65.95281 24.28177 F    A   TRUE
#> 5 53.29508 24.59924 F    A  FALSE
#> 6 41.79532 27.85067 M    C   TRUE

```

Characterizing the Missingness Structure

Before imputing, it is good practice to understand the extent and pattern of missingness. `describe()` returns a summary object that reports per-variable missingness proportions, complete-case counts, and the missingness pattern matrix.

Formally, let X be an $n \times p$ data matrix and define the missingness indicator

$$R_{ij} = \begin{cases} 1, & X_{ij} \text{ is missing,} \\ 0, & X_{ij} \text{ is observed.} \end{cases}$$

The per-variable missingness proportion is

$$\hat{\pi}_j = \frac{1}{n} \sum_{i=1}^n R_{ij},$$

and the complete-case count is

$$n_{cc} = \sum_{i=1}^n I(R_{i1} = 0, \dots, R_{ip} = 0).$$

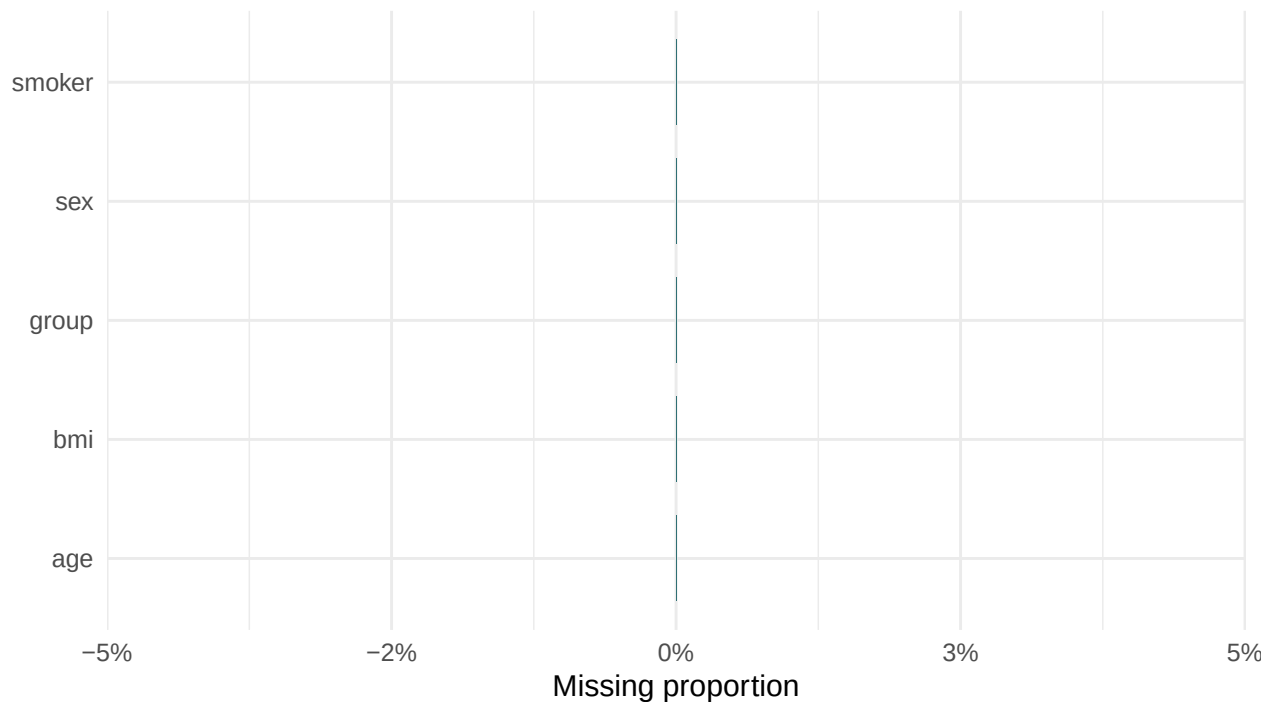
These summaries answer the first practical question: how much information would be lost under complete-case analysis, and which variables are driving that loss?

```

d <- describe(dat)
d
#> mimar missing-data description
#> Rows: 120 Columns: 5
#> Complete cases: 120 (100.0%)
summary(d)
#> # A tibble: 5 x 4
#>   variable type      n_missing prop_missing
#>   <chr>      <chr>      <dbl>      <dbl>
#> 1 age      numeric          0          0
#> 2 bmi      numeric          0          0
#> 3 sex      factor          0          0
#> 4 group    factor          0          0
#> 5 smoker   logical          0          0

```

```
plot(d)
```



Imputer Registry

`mimar` ships with native imputers and learner-backed methods. The registry documents each method, whether it is implemented natively or through a backend package, and which target types it supports.

```
imputer_registry()
#> # A tibble: 23 x 10
#>   imputer implementation package supports_numeric supports_binary
#>   <chr>      <chr>          <chr>      <lgl>          <lgl>
#> 1 mean      mimar          internal TRUE          TRUE
#> 2 median    mimar          internal TRUE          TRUE
#> 3 mode      mimar          internal TRUE          TRUE
#> 4 naive     mimar          internal TRUE          TRUE
#> 5 norm      mimar          internal TRUE          TRUE
#> 6 pmm       mimar          internal TRUE          TRUE
#> 7 spmm      mimar          internal TRUE          TRUE
#> 8 logreg    mimar          internal TRUE          TRUE
#> 9 polyreg   mimar          internal TRUE          TRUE
#> 10 rf       wrapped         ranger    TRUE          TRUE
#> # i 13 more rows
#> # i 5 more variables: supports_multiclass <lgl>, stochastic <lgl>,
#> #   description <chr>, available <lgl>, status <chr>
```

`describe("imputers")` returns the same registry through the unified `describe` interface, which also exposes the corresponding print and plot methods.

```
describe("imputers")
#> mimar available imputers
#> # A tibble: 23 x 10
#>   imputer implementation package supports_numeric supports_binary
```

```

#>   <chr>   <chr>           <chr>   <lgl>           <lgl>
#> 1 mean   mimar          internal TRUE          TRUE
#> 2 median mimar          internal TRUE          TRUE
#> 3 mode   mimar          internal TRUE          TRUE
#> 4 naive  mimar          internal TRUE          TRUE
#> 5 norm   mimar          internal TRUE          TRUE
#> 6 pmm    mimar          internal TRUE          TRUE
#> 7 spmm   mimar          internal TRUE          TRUE
#> 8 logreg mimar          internal TRUE          TRUE
#> 9 polyreg mimar          internal TRUE          TRUE
#> 10 rf     wrapped       ranger    TRUE          TRUE
#> # i 13 more rows
#> # i 5 more variables: supports_multiclass <lgl>, stochastic <lgl>,
#> #   description <chr>, available <lgl>, status <chr>

```

Simulating Missingness for Benchmarking

When the goal is to evaluate imputation quality, a natural approach is to start with a complete dataset, introduce missingness under a known mechanism, impute, and compare the recovered values to the truth. `ampute()` supports this workflow by generating artificial missingness and tracking three masks:

- `mask_original`: cells already missing before amputation;
- `mask_added`: cells made missing by `ampute()`;
- `mask_total`: all missing cells after amputation.

Only `mask_added` cells have known truth and are used by `evaluate()`.

Missing completely at random

Under MCAR, each cell in the target variable is independently assigned to be missing with constant probability:

$$A_{ij} \sim \text{Bernoulli}(\pi), \quad X_{ij} \leftarrow \text{NA if } A_{ij} = 1,$$

where π is `prop`. MCAR is the only mechanism under which complete-case analysis is unbiased, making it a useful baseline when comparing imputers.

Missing at random

Under MAR, the probability of missingness depends on observed covariates but not on the missing value itself. `mimar` constructs a standardized linear score s_i from the `by` variables and maps it through a logistic function:

$$p_i = \frac{1}{1 + \exp[-(\alpha + s_i)]}.$$

The intercept α is calibrated so that $n^{-1} \sum_i p_i \approx \text{prop}$. This ensures that the targeted missingness proportion is respected on average while allowing individual probabilities to vary with the observed predictors.

Missing not at random

Under MNAR, s_i is derived from the target variable itself, which is the mechanism most harmful to inference because the missingness is informative about the value being missed. For numeric targets, `ampute()` can emphasize the right tail, left tail, or both tails of the distribution.

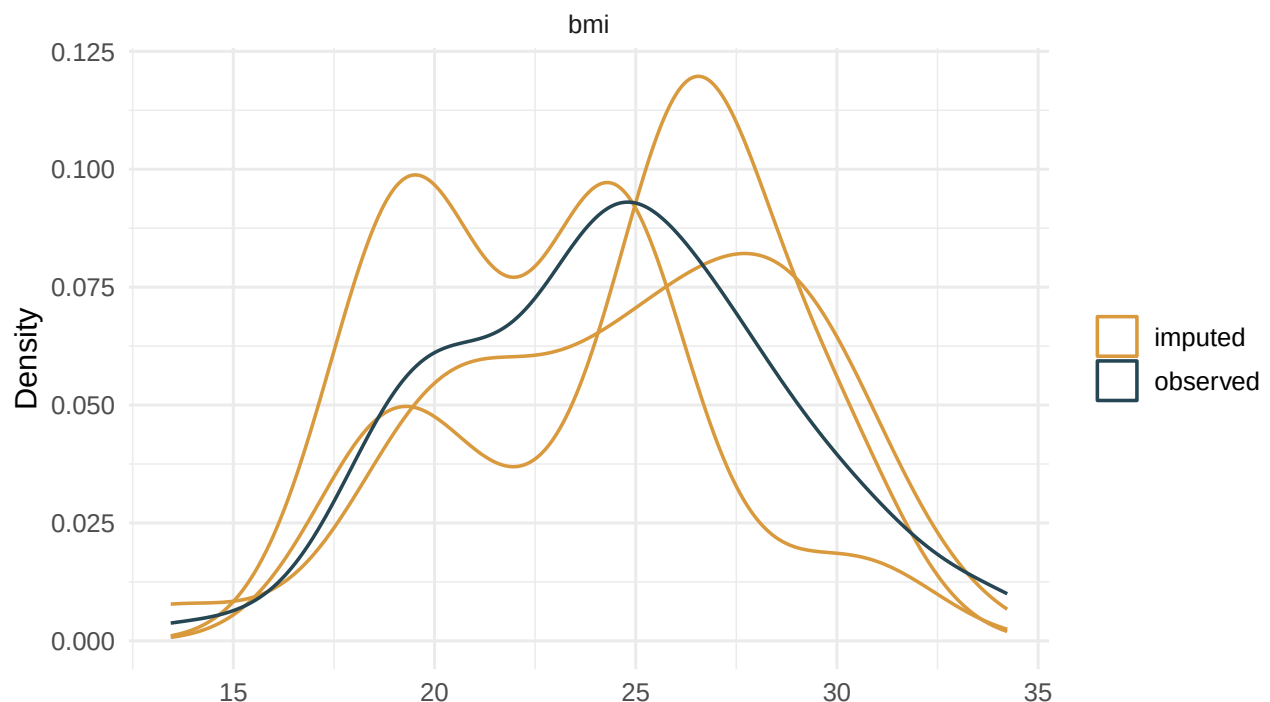
```
a <- ampute(
  dat,
  prop      = 0.25,
  mechanism = "MAR",
  target    = c("bmi", "group"),
  by        = c("age", "sex"),
  seed      = 1
)

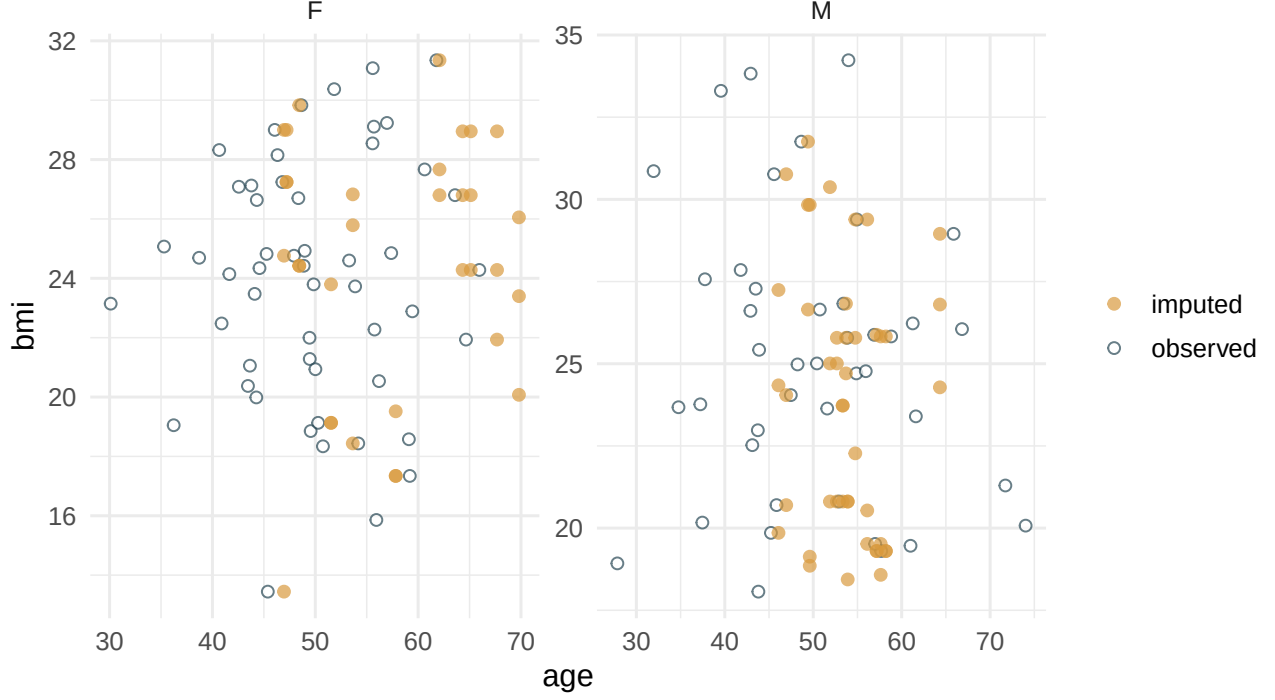
a
#> mivar amputation
#> Mechanism: MAR Added missing cells: 53
summary(a)
#> # A tibble: 5 x 4
#>   variable original added total
#>   <chr>         <dbl> <dbl> <dbl>
#> 1 age           0      0      0
#> 2 bmi           0     26     26
#> 3 sex           0      0      0
#> 4 group         0     27     27
#> 5 smoker        0      0      0
```

Quick Visual Checks

The first thing most users want to see is whether the diagnostics are actually visible and legible. This compact set of plots appears early in the vignette so the rendered HTML shows the core visual outputs without requiring a long scroll.

```
i_knn <- impute(a, imputer = "knn", m = 3, maxit = 3, seed = 1)
plot(i_knn, type = "density", variable = "bmi")
plot(i_knn, type = "xy", formula = bmi ~ age | sex)
```





The Chained Imputation Algorithm

The fully conditional specification approach imputes each incomplete variable in turn, conditioning on all other variables. This is repeated for T iterations and replicated m times to obtain m independent completed datasets.

For incomplete variable X_j , let

$$O_j = \{i : R_{ij} = 0\}, \quad M_j = \{i : R_{ij} = 1\}.$$

At iteration t , with the current completed data $\tilde{X}^{(t)}$, `mimr` fits an imputation model using the observed rows of variable j :

$$\hat{f}_j^{(t)} : \tilde{X}_{-j,O_j}^{(t)} \rightarrow X_{j,O_j}.$$

Missing values are then updated by applying the learned model to the missing rows:

$$\tilde{X}_{j,M_j}^{(t+1)} = g_j\left(\hat{f}_j^{(t)}, \tilde{X}_{-j,M_j}^{(t)}\right),$$

where g_j is the prediction or donor-sampling step specific to the chosen imputer. Observed values are never overwritten; $\tilde{X}_{j,O_j}^{(t)}$ is reset to X_{j,O_j} after each update.

The full algorithm, written compactly, is as follows. Let h be the imputer, T the number of chained iterations, m the number of imputations, and $\mathcal{B}(O_j)$ a bootstrap sample from the observed rows of variable j .

Input: X, R, h, m, T .
Initialize: $\tilde{X}^{(0)} \leftarrow \text{init}(X)$, medians for numeric, modes for categorical variables.
For $k = 1, \dots, m$: $\tilde{X}_k^{(0)} \leftarrow \tilde{X}^{(0)}$.
For $t = 1, \dots, T$:
 For each j with $M_j \neq \emptyset$:
 $B_j \leftarrow \mathcal{B}(O_j)$.
 $\hat{f}_{jk}^{(t)} \leftarrow \text{fit}_h(\tilde{X}_{k,B_j,-j}^{(t-1)}, X_{B_j,j})$.
 $\tilde{X}_{k,M_j,j}^{(t)} \leftarrow \text{restore}_j[g_h(\hat{f}_{jk}^{(t)}, \tilde{X}_{k,M_j,-j}^{(t-1)})]$.
 $\tilde{X}_{k,O_j,j}^{(t)} \leftarrow X_{O_j,j}$.
Return: $\{\tilde{X}_1^{(T)}, \dots, \tilde{X}_m^{(T)}\}$ as a `mimar_imputation` object.

The bootstrap draw within each iteration is what introduces between-imputation variability and ensures that the m completed datasets are not identical. The `restore` step converts predictions back to the target storage type (numeric, integer, `Date`, logical, factor, or character), which allows the same loop to handle numeric-only, categorical-only, and mixed-type data frames without special-casing at the call site.

Implemented Update Equations

The imputers differ in how $\hat{f}_j$ is fitted and how g_j generates draws. The choice of imputer determines both the distributional assumptions made about each variable and the computational cost per iteration.

Mean, Median, Mode, and Naive

These constant-predictor imputers ignore the predictor variables entirely. For numeric targets, `mean` and `median` use

$$\tilde{x}_{ij} = \bar{x}_j \quad \text{or} \quad \tilde{x}_{ij} = \text{median}(x_{O_j,j}).$$

For categorical targets, `mode` imputes the most frequent observed category:

$$\tilde{x}_{ij} = \arg \max_c \sum_{i \in O_j} I(x_{ij} = c).$$

`naive` selects median for numeric-like variables and mode for categorical-like variables automatically. These methods are primarily useful as baselines and for initialising the chain; they do not preserve the conditional distribution of the missing variable given observed covariates.

Normal Linear Draw

`norm` fits an ordinary linear model

$$X_j = X_{-j}\beta_j + \epsilon_j, \quad \epsilon_j \sim N(0, \sigma_j^2),$$

and draws from the predictive distribution at missing rows:

$$\tilde{X}_{j,M_j} = \hat{X}_{-j,M_j}\hat{\beta}_j + e, \quad e \sim N(0, \hat{\sigma}_j^2).$$

The stochastic draw is important: deterministic regression imputation underestimates the variance of the imputed variable and collapses the distribution around the regression surface.

Predictive Mean Matching

pmm combines linear prediction with donor sampling, which addresses the main limitation of **norm** for non-Gaussian variables: imputed values are constrained to the observed support. The method fits the same linear model but instead of drawing from the normal predictive distribution, it selects a donor from the observed rows whose predicted value is closest to the missing row's prediction:

$$d(i, \ell) = |\hat{\mu}_i - \hat{\mu}_\ell|, \quad \tilde{x}_{ij} = x_{\ell j}, \quad \ell \sim \mathcal{D}_i.$$

This makes PMM robust to departures from normality and ensures that imputed values respect the range and granularity of the observed data (Buuren and Groothuis-Oudshoorn 2011). **spmm** uses the same donor-matching rule in a single-step variant.

Logistic and Polytomous Regression

For binary targets, **logreg** models the conditional probability directly:

$$\Pr(X_j = 1 \mid X_{-j}) = \text{logit}^{-1}(X_{-j}\beta_j),$$

and imputes by drawing

$$\tilde{X}_{ij} \sim \text{Bernoulli}(\hat{p}_i).$$

For multiclass targets, **polyreg** fits one-vs-rest logistic models for each class $c = 1, \dots, C$ and normalises the resulting probabilities:

$$\tilde{p}_{ic} = \frac{\hat{p}_{ic}}{\sum_{r=1}^C \hat{p}_{ir}},$$

then draws from $\text{Categorical}(\tilde{p}_i)$. Stochastic class draws, rather than hard majority-class assignment, are necessary for preserving the uncertainty that multiple imputation is designed to propagate.

Donor Methods: knn, hotdeck, famd

knn and **hotdeck** are donor-based imputers that do not assume any parametric form for the conditional distribution. Predictors are encoded into a numeric design matrix Z . The donor distance for missing row i is

$$d(i, \ell) = \|Z_i - Z_\ell\|_2.$$

knn samples the imputed value from the k nearest observed donors. **hotdeck** uses the same standardized distance inside the chained-equation loop, operating as a stochastic hot-deck update. **famd** extends the donor approach to mixed-type predictors by first projecting observations into FAMD-assisted coordinates with **missMDA** (Josse and Husson 2016), then applying donor matching in that space.

Learner-Backed Imputers

rf, **ranger**, **rpart**, **nbayes**, **svm**, **bart**, **glmnet**, **gbm**, and **xgboost** treat the imputation of each variable as a supervised learning problem. The learner is fitted on observed rows and predicts (or draws from) the conditional distribution at missing rows:

$$\hat{f}_j = \mathcal{A}_h(X_{-j, O_j}, X_{j, O_j}).$$

Numeric targets use regression predictions; binary and multiclass targets use sampled class probabilities when the backend exposes them. These learner-backed methods are pragmatic stochastic chained update rules. They are useful for predictive recovery and sensitivity analyses, but users should inspect between-imputation variability, convergence-screening plots, and downstream model sensitivity rather than assuming that every learner automatically yields proper multiple-imputation uncertainty. `rf` operates as a variable-level supervised update inside `mimar`'s chained loop; it is not a whole-dataframe imputation engine. This is conceptually related to the `missForest` strategy (Stekhoven and Bühlmann 2012), but the outer iteration, multiple-imputation replication, and type handling are controlled by `mimar`. The `ranger`, `glmnet`, `gbm`, and `xgboost` backends follow Wright and Ziegler (2017), J. Friedman, Hastie, and Tibshirani (2010), J. H. Friedman (2001), and Chen and Guestrin (2016), respectively.

Running the Imputation

All imputers are called through the same `impute()` interface. The `m` argument controls the number of completed datasets and `maxit` the number of chained-equation cycles per dataset. Increasing `maxit` allows the conditional distributions more cycles to converge; in practice, 5 to 10 iterations is often sufficient for well-behaved data, though trace diagnostics and sensitivity analyses should be checked for complex missingness patterns.

When a learner-backed imputer is requested, `mimar` applies that imputer to each incomplete variable rather than silently substituting another method. This keeps benchmark comparisons interpretable: `imputer = "rf"` means that random-forest updates are used for numeric, binary, and multiclass targets, and the same principle holds for the other learner-backed imputers.

```
i_knn <- impute(a, imputer = "knn", m = 3, maxit = 3, seed = 1)
i_knn
#> mimar imputation
#> Completed datasets: 3
#> Rows x columns: 120 x 5
#> Imputer: knn maxit: 3 ncore: 1 stochastic: TRUE
#> Missing cells imputed: 53 / 53
#> Variables imputed: bmi, group
#> Use complete(x) to extract completed data.
summary(i_knn)
#> mimar imputation summary
#> # A tibble: 1 x 11
#>   rows columns n_imputations imputer maxit ncore stochastic
#>   <int>   <int>         <int> <chr>   <dbl> <int> <lgl>
#> 1   120       5             3 knn      3     1 TRUE
#> # i 4 more variables: total_missing_before <int>, total_imputed <int>,
#> #   remaining_missing <int>, variables_imputed <int>
#>
#> Variables:
#> # A tibble: 5 x 9
#>   variable type      method n_missing_before prop_missing_before n_imputed
#>   <chr>      <chr>    <chr>          <int>          <dbl>          <int>
#> 1 age      numeric none              0              0              0
#> 2 bmi      numeric knn              26             0.217          26
#> 3 sex      factor  none              0              0              0
#> 4 group    factor  knn              27             0.225          27
#> 5 smoker   logical none              0              0              0
#> # i 3 more variables: prop_imputed <dbl>, remaining_missing <int>,
#> #   between_imputation_sd <dbl>
complete(i_knn, 1)
#> # A tibble: 120 x 5
```

```

#>      age    bmi sex  group smoker
#>    <dbl> <dbl> <fct> <fct> <lgl>
#>  1  43.7  23.0 M     C     FALSE
#>  2  51.8  30.4 F     A     FALSE
#>  3  41.6  24.1 F     B     TRUE
#>  4  66.0  24.3 F     C     TRUE
#>  5  53.3  24.6 F     A     FALSE
#>  6  41.8  27.9 M     C     TRUE
#>  7  54.9  24.7 M     B     TRUE
#>  8  57.4  24.8 F     B     FALSE
#>  9  55.8  22.3 F     B     FALSE
#> 10  46.9  30.8 M     A     FALSE
#> # i 110 more rows

```

The vignette uses `knn` for the main diagnostics because it is dependency-light, fast on the small example data, and produces visible stochastic donor variation across imputations. The same plotting calls work for the other registered imputers.

The `m` completed datasets are conditionally independent once their random seeds have been fixed. `mimr` therefore parallelises at the completed-dataset level when `ncore > 1`. Internally, the package maps the same single-chain imputation routine over `1:m` with `functionals::fmap()`. This is the natural parallel boundary: each worker receives the original incomplete data, the initial median/mode completion, the imputer specification, and a deterministic seed offset. Worker `k` uses `seed + k - 1`, so a run remains reproducible for a fixed `seed`, `m`, `maxit`, and imputer.

```

i_parallel <- impute(
  a,
  imputer = "knn",
  m       = 5,
  maxit   = 5,
  seed    = 1,
  ncore   = 2
)

```

The default `ncore = 1` is deliberately conservative. It avoids parallel overhead for small data and gives the most predictable behavior in constrained execution environments. Increasing `ncore` is most useful when `m` is greater than one and each chain is expensive, for example with random forests, gradient boosting, penalized regression, or BART. Parallelism does not change the chained-equation model; it only changes how the independent completed datasets are scheduled.

Methods that differ in their statistical assumptions can be compared by running them on the same amputed object:

```

i_knn_small <- impute(a, imputer = "knn", m = 1, maxit = 2, seed = 1)
i_hotdeck <- impute(a, imputer = "hotdeck", m = 1, maxit = 2, seed = 1)

```

```

summary(i_knn_small)
#> mimar imputation summary
#> # A tibble: 1 x 11
#>   rows columns n_imputations imputer maxit ncore stochastic
#>   <int>   <int>          <int> <chr>   <dbl> <int> <lgl>
#> 1   120     5              1 knn      2     1 TRUE
#> # i 4 more variables: total_missing_before <int>, total_imputed <int>,
#> #   remaining_missing <int>, variables_imputed <int>
#>
#> Variables:

```

```

#> # A tibble: 5 x 9
#>   variable type      method n_missing_before prop_missing_before n_imputed
#>   <chr>      <chr>    <chr>          <int>          <dbl>          <int>
#> 1 age        numeric none              0              0              0
#> 2 bmi        numeric knn              26             0.217          26
#> 3 sex        factor  none              0              0              0
#> 4 group      factor  knn              27             0.225          27
#> 5 smoker     logical none              0              0              0
#> # i 3 more variables: prop_imputed <dbl>, remaining_missing <int>,
#> #   between_imputation_sd <dbl>
summary(i_hotdeck)
#> mivar imputation summary
#> # A tibble: 1 x 11
#>   rows columns n_imputations imputer maxit ncore stochastic
#>   <int>   <int>      <int> <chr>   <dbl> <int> <lgl>
#> 1   120       5            1 hotdeck  2     1 TRUE
#> # i 4 more variables: total_missing_before <int>, total_imputed <int>,
#> #   remaining_missing <int>, variables_imputed <int>
#>
#> Variables:
#> # A tibble: 5 x 9
#>   variable type      method n_missing_before prop_missing_before n_imputed
#>   <chr>      <chr>    <chr>          <int>          <dbl>          <int>
#> 1 age        numeric none              0              0              0
#> 2 bmi        numeric hotdeck          26             0.217          26
#> 3 sex        factor  none              0              0              0
#> 4 group      factor  hotdeck          27             0.225          27
#> 5 smoker     logical none              0              0              0
#> # i 3 more variables: prop_imputed <dbl>, remaining_missing <int>,
#> #   between_imputation_sd <dbl>

```

Learner-backed methods are available through the same call:

```

impute(a, imputer = "rf",      m = 5, maxit = 5, seed = 1)
impute(a, imputer = "glmnet", m = 5, maxit = 5, seed = 1)
impute(a, imputer = "xgboost", m = 5, maxit = 5, seed = 1)
impute(a, imputer = "superlearner", m = 5, maxit = 5, seed = 1)

```

Tuning Imputers

Learner-backed imputers expose their hyperparameters through a reusable specification object, or directly through ... when calling `impute()`. The same hyperparameter set is applied across the full chained-imputation pipeline.

```

rf_spec <- imputer("rf", num.trees = 500)
xgb_spec <- imputer("xgboost", nrounds = 100, max_depth = 3)

describe(a)
#> mivar missing-data description
#> Rows: 120 Columns: 5
#> Complete cases: 78 (65.0%)

i_knn <- impute(a, imputer = "knn", m = 3, maxit = 3, seed = 1, donors = 10)

summary(i_knn)

```

```
#> mivar imputation summary
#> # A tibble: 1 x 11
#>   rows columns n_imputations imputer maxit ncore stochastic
#>   <int>   <int>         <int> <chr>   <dbl> <int> <lgl>
#> 1   120     5             3 knn      3     1 TRUE
#> # i 4 more variables: total_missing_before <int>, total_imputed <int>,
#> #   remaining_missing <int>, variables_imputed <int>
#>
#> Variables:
#> # A tibble: 5 x 9
#>   variable type      method n_missing_before prop_missing_before n_imputed
#>   <chr>      <chr>    <chr>          <int>                <dbl>      <int>
#> 1 age      numeric none              0                    0          0
#> 2 bmi      numeric knn              26                   0.217      26
#> 3 sex      factor  none              0                    0          0
#> 4 group    factor  knn              27                   0.225      27
#> 5 smoker   logical none              0                    0          0
#> # i 3 more variables: prop_imputed <dbl>, remaining_missing <int>,
#> #   between_imputation_sd <dbl>
```

The learner-backed specifications can then be used in the same pipeline:

```
i_rf <- impute(a, imputer = rf_spec, m = 3, maxit = 3, seed = 1)
i_xgb <- impute(a, imputer = xgb_spec, m = 3, maxit = 3, seed = 1)
```

Super Learner Imputation

`superlearner` builds a cross-validated ensemble of candidate imputers. For each incomplete variable, the candidate library is evaluated on observed cells, non-negative weights are assigned from the observed-cell prediction loss, and the weighted ensemble is used inside the same chained-imputation loop.

```
sl_spec <- imputer(
  "superlearner",
  library = c("pmm", "knn", "rpart"),
  folds = 3,
  metalearner = "inverse_loss"
)

i_sl <- impute(a, imputer = sl_spec, m = 2, maxit = 2, seed = 1)
summary(i_sl)
#> mivar imputation summary
#> # A tibble: 1 x 11
#>   rows columns n_imputations imputer      maxit ncore stochastic
#>   <int>   <int>         <int> <chr>        <dbl> <int> <lgl>
#> 1   120     5             2 superlearner  2     1 TRUE
#> # i 4 more variables: total_missing_before <int>, total_imputed <int>,
#> #   remaining_missing <int>, variables_imputed <int>
#>
#> Variables:
#> # A tibble: 5 x 9
#>   variable type      method      n_missing_before prop_missing_before n_imputed
#>   <chr>      <chr>    <chr>          <int>                <dbl>      <int>
#> 1 age      numeric none              0                    0          0
#> 2 bmi      numeric superlearner  26                   0.217      26
#> 3 sex      factor  none              0                    0          0
```

```
#> 4 group      factor superlearner      27      0.225      27
#> 5 smoker     logical none          0          0          0
#> # i 3 more variables: prop_imputed <dbl>, remaining_missing <int>,
#> #   between_imputation_sd <dbl>
```

The shorter `sl` alias is equivalent:

```
impute(a, imputer = "sl", m = 5, maxit = 5, seed = 1)
```

Evaluating Imputation Quality

When the imputation object was produced from an amputated dataset, the true values of the artificially introduced missing cells are known. `evaluate()` exploits this by scoring only cells where `mask_added == TRUE`, isolating imputation error from any pre-existing missingness.

For numeric variables with true values y_i and imputed values $\hat{y}_i$ over the q recovered cells:

$$\text{RMSE} = \sqrt{\frac{1}{q} \sum_{i=1}^q (\hat{y}_i - y_i)^2}, \quad \text{MAE} = \frac{1}{q} \sum_{i=1}^q |\hat{y}_i - y_i|, \quad \text{Bias} = \frac{1}{q} \sum_{i=1}^q (\hat{y}_i - y_i).$$

RMSE penalises large errors more heavily than MAE; comparing the two gives a sense of whether the imputer is making a few large mistakes or many small ones. Bias diagnoses systematic over- or under-imputation, which can occur when the imputation model is misspecified relative to the true conditional distribution.

For categorical variables,

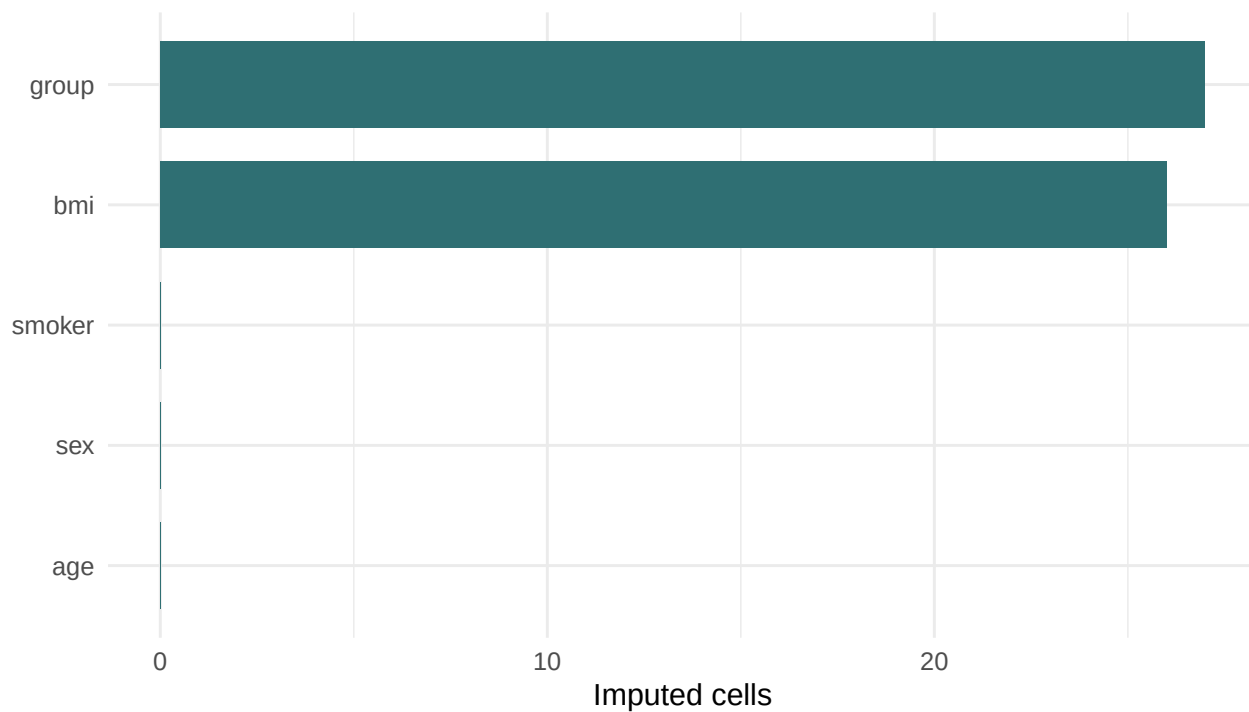
$$\text{Accuracy} = \frac{1}{q} \sum_{i=1}^q I(\hat{y}_i = y_i).$$

Balanced accuracy averages per-class recall and is more informative when class frequencies are unequal among the missing cells.

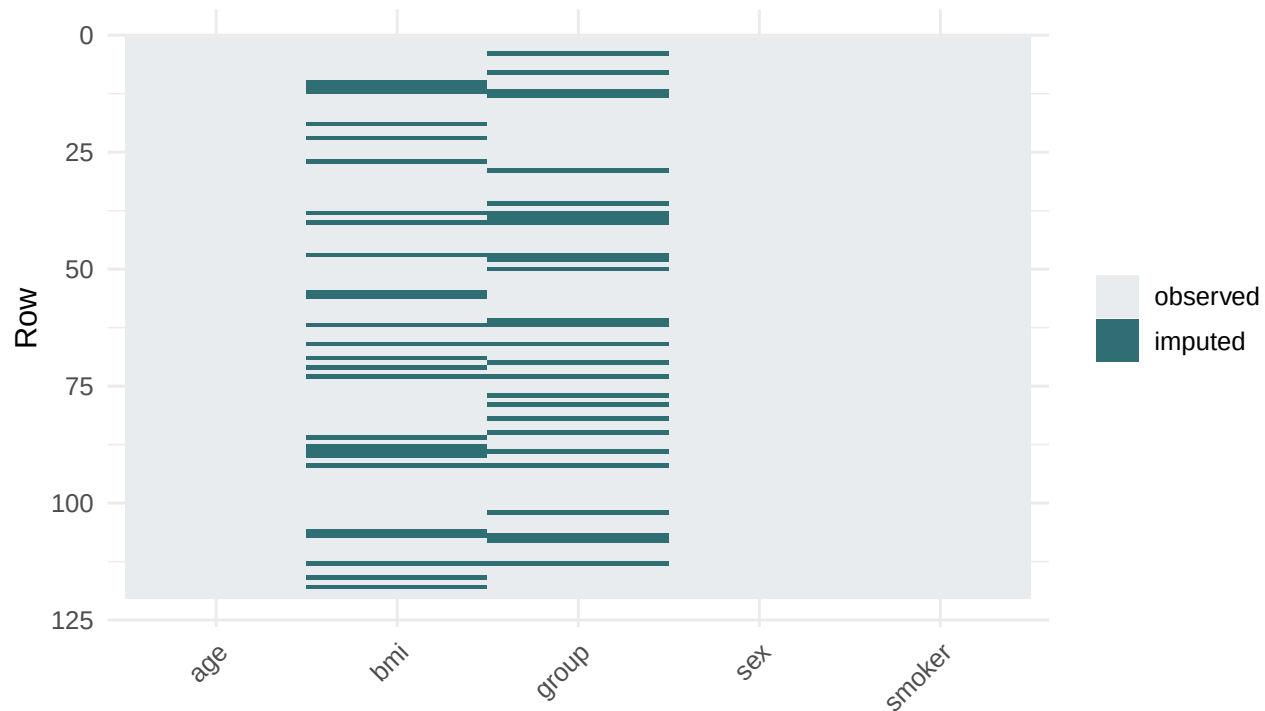
```
e <- evaluate(i_knn)
e
#> mivar imputation evaluation
#> # A tibble: 1 x 4
#>   n_imputations imputer total_missing evaluated_cells
#>   <int> <chr>          <int>          <int>
#> 1           3 knn             53             53
describe(e)
#> # A tibble: 1 x 4
#>   n_imputations imputer total_missing evaluated_cells
#>   <int> <chr>          <int>          <int>
#> 1           3 knn             53             53
head(e$recovery_by_imputation)
#> # A tibble: 6 x 9
#>   imputation variable type      rmse  mae  bias correlation accuracy
#>   <int> <chr>      <chr>    <dbl> <dbl> <dbl>      <dbl>    <dbl>
#> 1     1     bmi    numeric    6.80  5.95 -0.270    -0.436    NA
#> 2     1  group    categorical NA    NA    NA        NA      0.333
#> 3     2     bmi    numeric    6.78  5.39 -2.05    -0.193    NA
#> 4     2  group    categorical NA    NA    NA        NA      0.333
#> 5     3     bmi    numeric    6.34  4.93 -2.50    -0.250    NA
```

```
#> 6      3 group  categorical NA    NA    NA      NA      0.444
#> # i 1 more variable: balanced_accuracy <dbl>
```

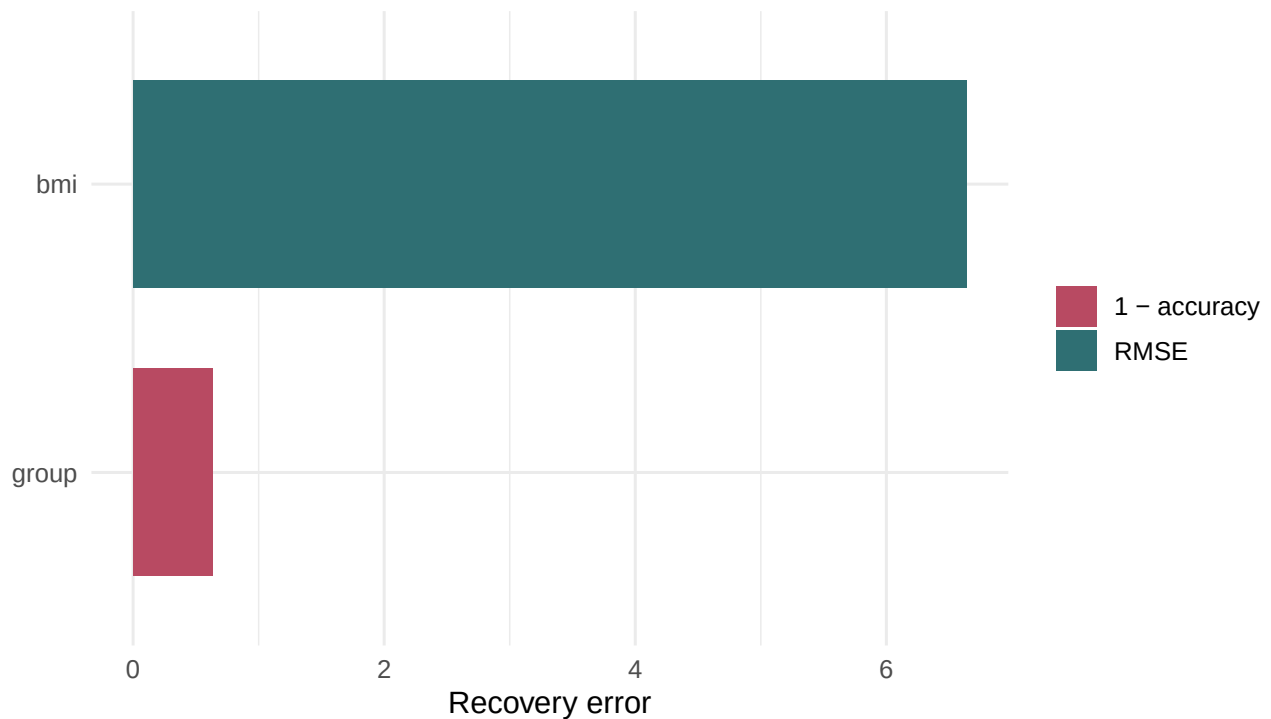
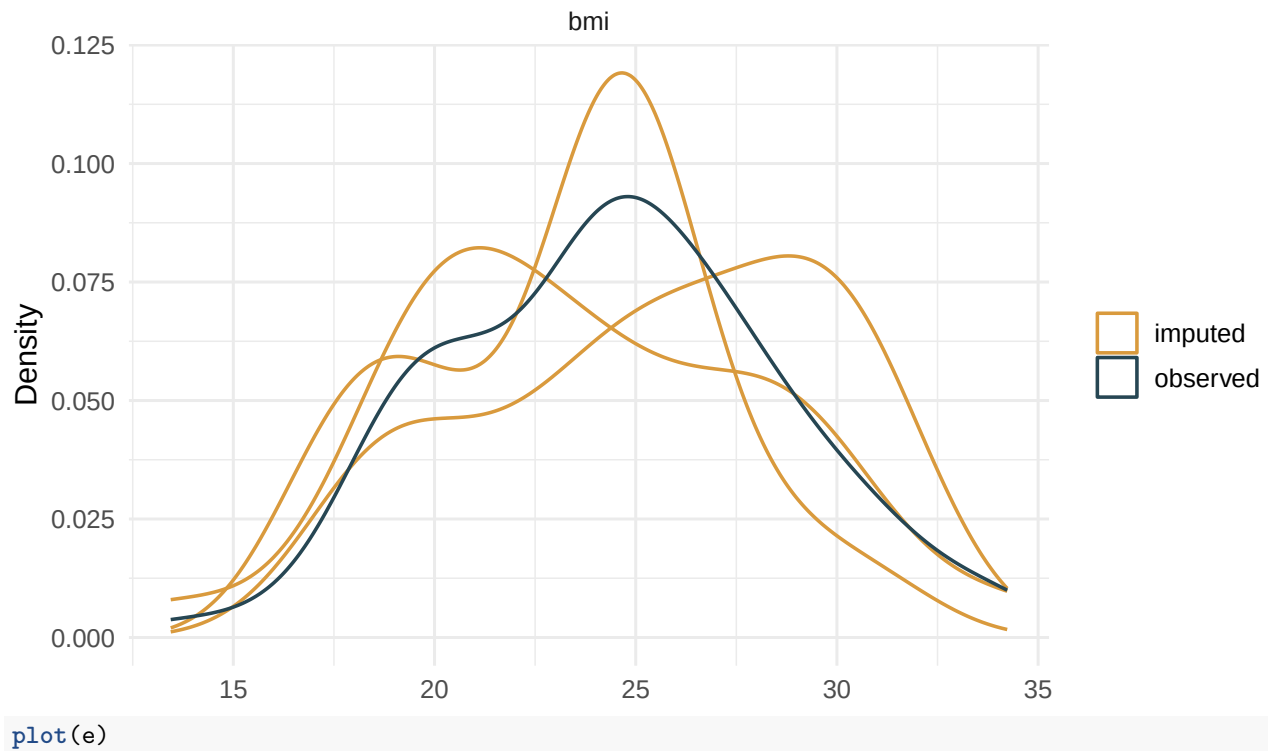
```
plot(i_knn)
```



```
plot(i_knn, type = "missing")
```



```
plot(i_knn, type = "density")
```



Diagnostic Plotting

Diagnostic plotting is not a cosmetic step in multiple imputation. The pooled analysis assumes that the imputed values are plausible draws from the conditional distribution implied by the imputation model. No single plot proves that assumption or establishes convergence, but several simple graphics expose common failures: chains that have not stabilised, imputed values with implausible marginal distributions, categorical levels that are over- or under-produced, and relationships between variables that are distorted among the

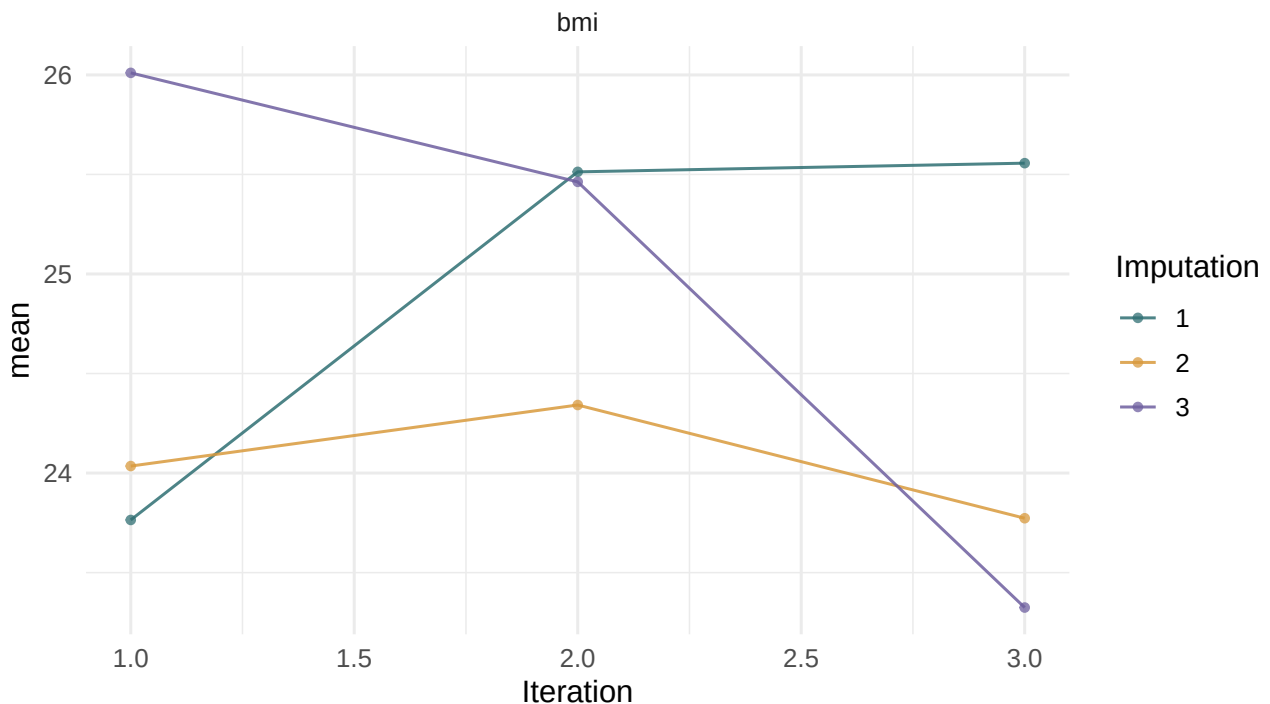
imputed cells.

`mimr` keeps lightweight iteration diagnostics in the imputation object. For each completed dataset, iteration, and incomplete variable, it records the number of imputed cells and, for numeric variables, the mean and standard deviation of the current imputed values. These summaries are small enough to store by default but rich enough to support convergence-screening trace plots.

```
head(i_knn$diagnostics$trace)
#> # A tibble: 6 x 9
#>   imputation iteration variable method type  n_imputed mean  sd unique
#>   <int>      <int> <chr>    <chr> <chr>    <int> <dbl> <dbl> <int>
#> 1         1         1  bmi     knn   numeric    26  23.8  4.42    19
#> 2         1         1  group   knn   factor    27  NA    NA      3
#> 3         1         2  bmi     knn   numeric    26  25.5  3.24    17
#> 4         1         2  group   knn   factor    27  NA    NA      3
#> 5         1         3  bmi     knn   numeric    26  25.6  4.30    21
#> 6         1         3  group   knn   factor    27  NA    NA      3
```

The trace plot shows how a summary of the imputed values changes over the chained iterations. Strong monotone trends or chains that remain separated can signal unstable chained updates, feedback between derived variables, or a poor predictor specification. Flat traces are not automatically good: deterministic or constant imputers such as `naive` can be flat because they do not generate meaningful stochastic updates. Trace plots are screening tools, not formal proofs of convergence, and are most informative for stochastic methods such as `pmm`, `norm`, `logreg`, `polyreg`, and learner-backed imputers.

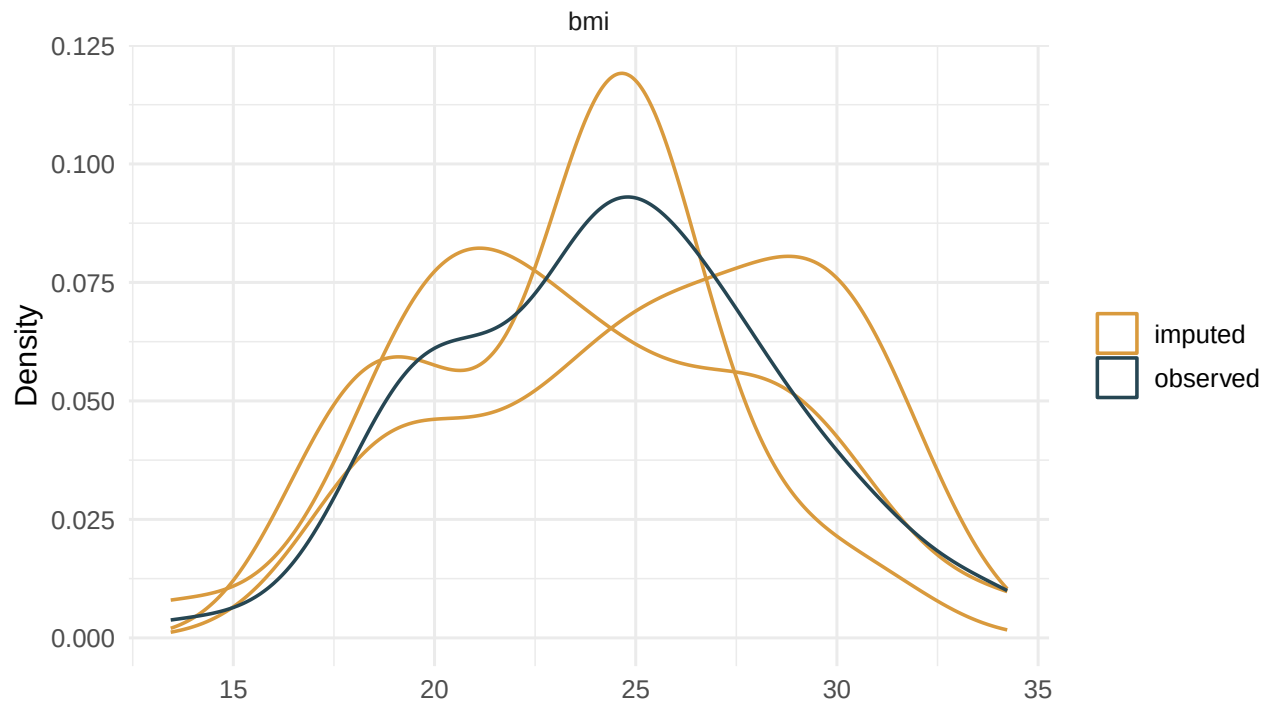
```
plot(i_knn, type = "trace", statistic = "mean")
```



Density plots compare the observed distribution with the distribution of imputed values. The observed data are drawn once, while imputed values are drawn separately for each completed dataset. This mirrors the common diagnostic style where the observed curve acts as a reference and each imputation contributes its own imputed curve. Density diagnostics are drawn as lines rather than filled areas so that several completed datasets can be compared without the later layers obscuring the earlier ones. The marginal distributions do not have to be identical under MAR, because missingness may depend on observed covariates. Large differences should still be investigated because they may represent either a real conditional difference or an

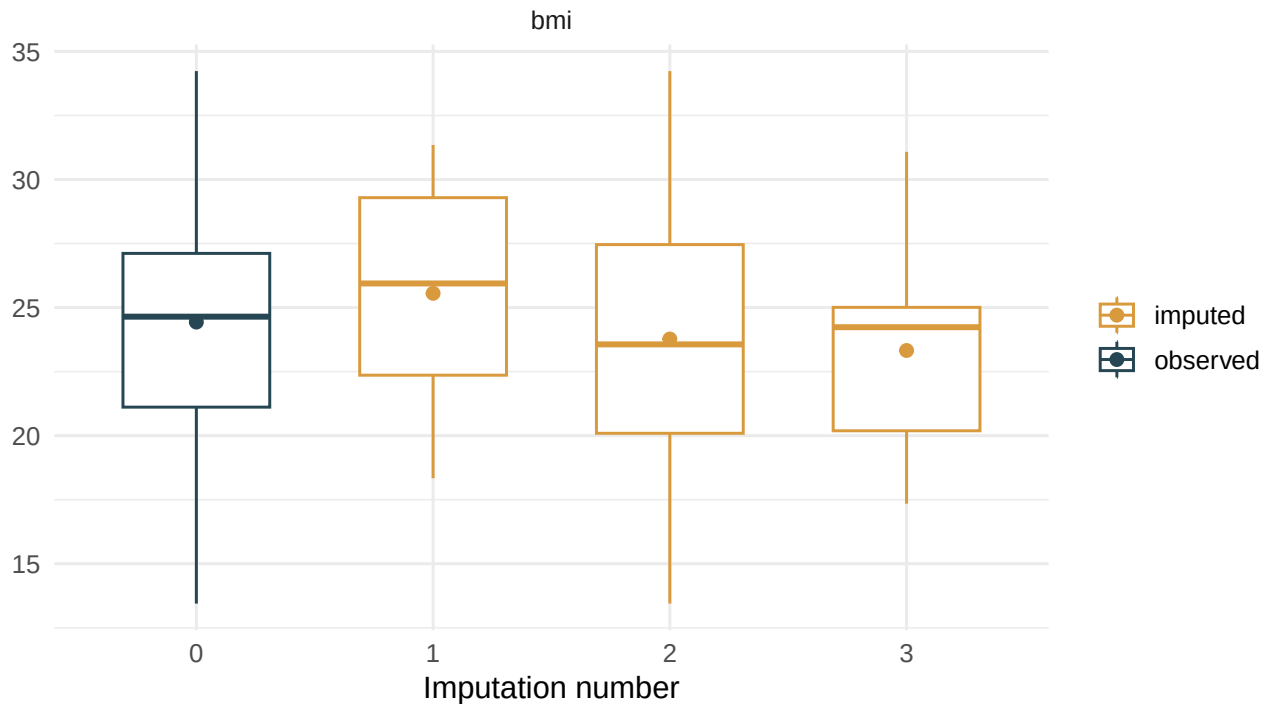
imputation-model problem.

```
plot(i_knn, type = "density", variable = "bmi")
```



Box-and-whisker diagnostics provide the same comparison in a more robust form. Imputation number 0 denotes the observed values and 1:m denote the completed datasets. This is useful when densities are unstable because there are few imputed values or because the variable has a skewed distribution with outliers.

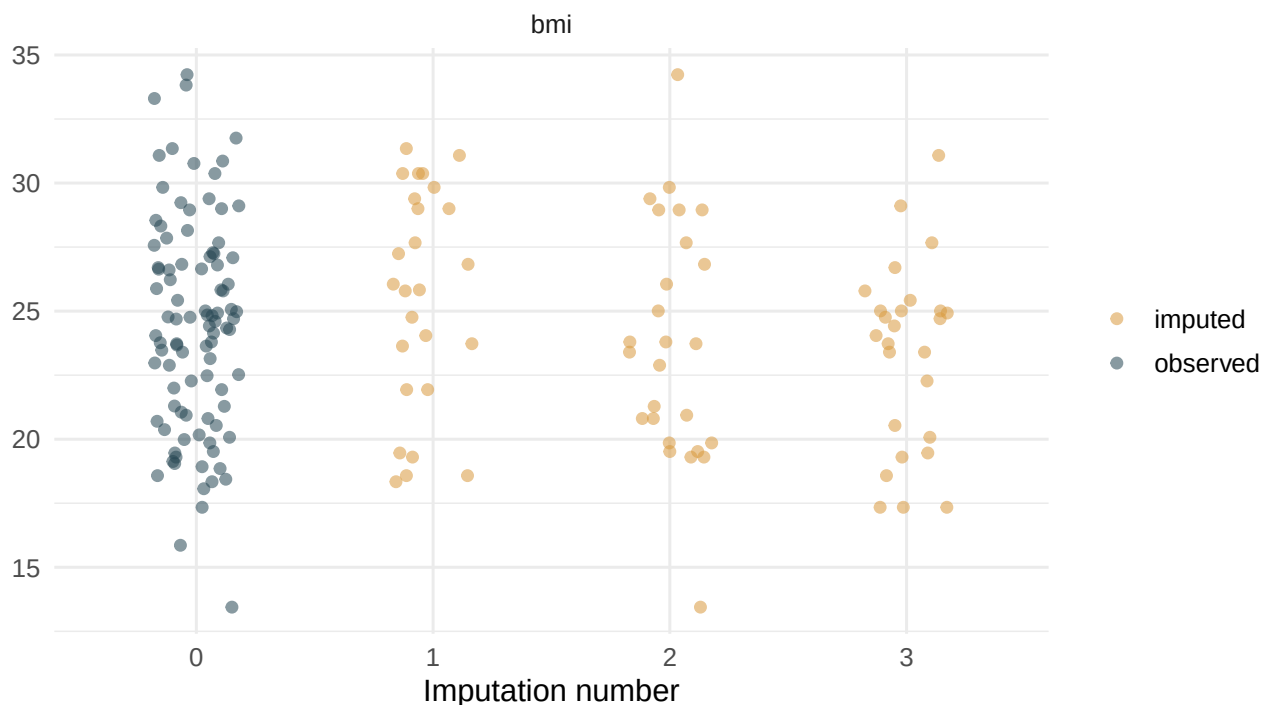
```
plot(i_knn, type = "boxplot", variable = "bmi")
```



The strip plot displays individual observed and imputed values by imputation number. It is especially helpful

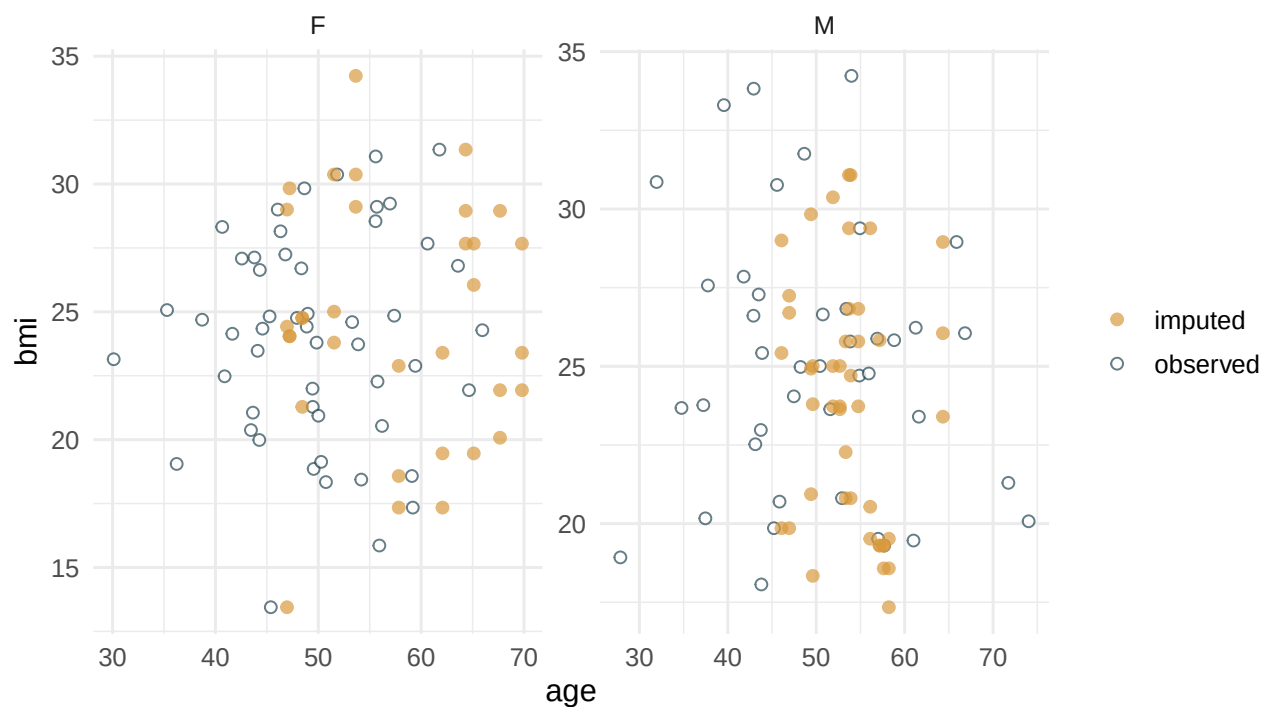
for small datasets, variables with few missing cells, and situations where overplotting is limited. A tight pile of imputed points at one value reveals deterministic or overly concentrated imputation.

```
plot(i_knn, type = "strip", variable = "bmi")
```



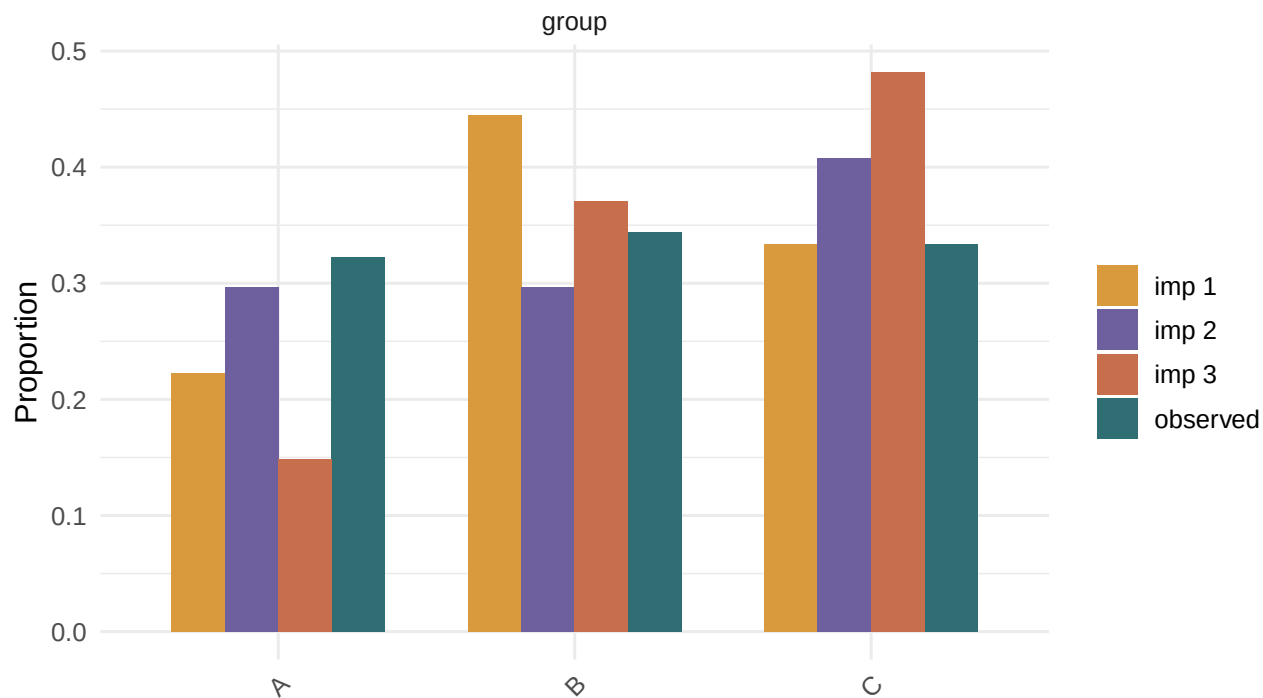
Bivariate diagnostics examine whether the imputed rows preserve relationships between variables. The formula interface follows the familiar $y \sim x$ pattern, with an optional conditioning variable after $|$. Observed points are shown separately from rows where either plotted variable had to be imputed.

```
plot(i_knn, type = "xy", formula = bmi ~ age | sex)
```

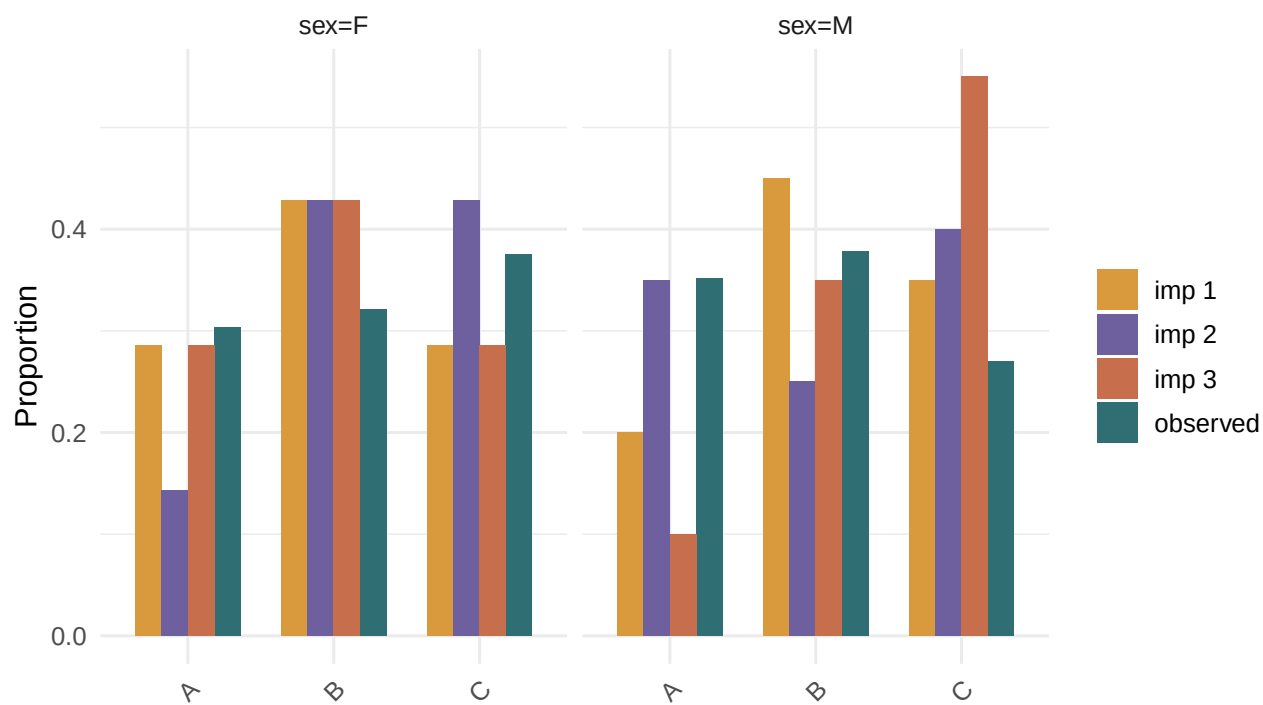


For categorical variables, density and boxplot diagnostics are not meaningful. The proportion plot compares the category distribution among observed values with the category distribution generated within each imputation. A stratified formula can be used when a categorical discrepancy may be explained by another observed variable.

```
plot(i_knn, type = "proportion", variable = "group")
```



```
plot(i_knn, type = "proportion", formula = group ~ sex)
```



Pooling Across Imputations

The purpose of generating $m > 1$ completed datasets is to obtain valid standard errors that reflect uncertainty about the missing data. Analysing each completed dataset separately and then combining results using Rubin's rules (Rubin 1987; Marshall et al. 2009) accomplishes this.

`pool()` is designed for post-fit quantities computed after imputation. The statistical target is the quantity, not the storage container. A quantity may be a scalar estimate, a vector of regression coefficients, a covariance-aware parameter vector, a matrix of survival probabilities, or a scalar performance metric. Data frames are accepted only as a tidy adapter for scalar estimates that are naturally stored one row per term and imputation.

For a single quantity, let Q_k be its estimate and U_k the complete-data variance estimate from completed dataset k . The pooled point estimate and between-imputation variance are:

$$\bar{Q} = \frac{1}{m} \sum_{k=1}^m Q_k, \quad B = \frac{1}{m-1} \sum_{k=1}^m (Q_k - \bar{Q})^2.$$

The total variance combines within- and between-imputation components:

$$\bar{U} = \frac{1}{m} \sum_{k=1}^m U_k, \quad T = \bar{U} + \left(1 + \frac{1}{m}\right) B.$$

The term $(1 + m^{-1})B$ is the excess variance attributable to missingness. The fraction of missing information, $\lambda = (1 + m^{-1})B/T$, quantifies how much the missingness inflates uncertainty relative to a complete-data analysis; variables with large λ are those for which inference is most sensitive to the imputation model.

The result is a `mimar_pool` object whose `pooled` table contains the pooled quantity, total standard error, confidence interval, and variance diagnostics for each `term`.

```
pool(c(0.10, 0.11, 0.09), std.error = c(0.04, 0.05, 0.04), name = "age")
#> mimar pooled results
#> # A tibble: 1 x 14
#>   term estimate std.error statistic    df p.value conf.low conf.high    m
#>   <chr>      <dbl>    <dbl>    <dbl> <dbl> <dbl>    <dbl>    <dbl> <int>
#> 1 age         0.1    0.0451     2.22  465.  0.0271  0.0114    0.189     3
#> # i 5 more variables: within_variance <dbl>, between_variance <dbl>,
#> #   total_variance <dbl>, relative_increase_variance <dbl>, rule <chr>
```

For several coefficients from the same fitted model, the vector and covariance matrix should be pooled together. This keeps the joint uncertainty structure, which is needed for multivariate Wald tests and downstream contrasts.

```
betas <- list(
  c(age = 0.10, bmi = 0.30),
  c(age = 0.11, bmi = 0.32),
  c(age = 0.09, bmi = 0.29)
)
covariances <- list(
  diag(c(0.04, 0.08)^2),
  diag(c(0.05, 0.09)^2),
  diag(c(0.04, 0.08)^2)
)

pooled_betas <- pool(betas, covariance = covariances)
pooled_betas
```

```

#> mimar pooled results
#> # A tibble: 2 x 14
#>   term estimate std.error statistic   df p.value conf.low conf.high   m
#>   <chr>      <dbl>    <dbl>    <dbl> <dbl> <dbl>    <dbl>    <dbl> <int>
#> 1 age       0.1      0.0451     2.22  465. 0.0271     0.0114     0.189     3
#> 2 bmi       0.303    0.0853     3.56 1094. 0.000393    0.136     0.471     3
#> # i 5 more variables: within_variance <dbl>, between_variance <dbl>,
#> #   total_variance <dbl>, relative_increase_variance <dbl>, rule <chr>
pooled_betas$estimate
#>      age      bmi
#> 0.1000000 0.3033333
pooled_betas$variance
#>      age      bmi
#> age 0.002033333 0.000200000
#> bmi 0.000200000 0.007277778

```

Matrix-valued quantities are supplied as a list with one matrix per imputation. For example, a survival probability matrix might have time points in rows and covariate profiles in columns. Unless a full joint covariance is available, `pool()` treats each cell as a scalar estimand and pools element by element.

```

survival_probabilities <- list(
  matrix(c(0.90, 0.80, 0.70, 0.60), nrow = 2),
  matrix(c(0.91, 0.79, 0.72, 0.61), nrow = 2),
  matrix(c(0.89, 0.81, 0.71, 0.59), nrow = 2)
)

pooled_survival <- pool(survival_probabilities)
pooled_survival
#> mimar pooled results
#> # A tibble: 4 x 14
#>   term estimate std.error conf.low conf.high   m mean median q25 q75
#>   <chr>      <dbl>    <dbl>    <dbl>    <dbl> <int> <dbl> <dbl> <dbl> <dbl>
#> 1 q[1,1]    0.9      NA      0.89     0.91     3 0.9   0.9  0.895 0.905
#> 2 q[2,1]    0.8      NA      0.79     0.81     3 0.8   0.8  0.795 0.805
#> 3 q[1,2]    0.71     NA      0.7      0.72     3 0.71  0.71 0.705 0.715
#> 4 q[2,2]    0.6      NA      0.59     0.61     3 0.6   0.6  0.595 0.605
#> # i 4 more variables: iqr <dbl>, min <dbl>, max <dbl>, rule <chr>
pooled_survival$estimate
#>      [,1] [,2]
#> [1,] 0.9 0.71
#> [2,] 0.8 0.60

```

The tidy data-frame adapter remains useful for model outputs that already have one scalar row per term and imputation. In the next example there are six input rows because two quantities, `age` and `bmi`, are estimated in each of three imputations. `pool()` returns two pooled quantities: one row for `age` and one row for `bmi`.

```

external_results <- data.frame(
  term      = rep(c("age", "bmi"), each = 3),
  estimate  = c(0.10, 0.11, 0.09, 0.30, 0.32, 0.29),
  std.error = c(0.04, 0.05, 0.04, 0.08, 0.09, 0.08),
  imputation = rep(1:3, times = 2)
)

p <- pool(external_results)
p

```

```
#> mivar pooled results
#> # A tibble: 2 x 14
#>   term estimate std.error statistic   df p.value conf.low conf.high   m
#>   <chr>      <dbl>      <dbl>      <dbl> <dbl>   <dbl>      <dbl>      <dbl> <int>
#> 1 age         0.1        0.0451        2.22  465. 0.0271      0.0114      0.189    3
#> 2 bmi         0.303      0.0853        3.56 1094. 0.000393    0.136      0.471    3
#> # i 5 more variables: within_variance <dbl>, between_variance <dbl>,
#> #   total_variance <dbl>, relative_increase_variance <dbl>, rule <chr>
```

When no reliable complete-data variance is available, as is common for bounded performance measures, Marshall et al. recommend robust summaries. Accordingly, `pool()` reports the median, interquartile range, and range by default for such quantities. Use `rule = "mean"` only when a mean and between-imputation standard error are substantively appropriate.

Limitations and Future Work

`mivar` is intentionally compact. It standardizes missingness description, amputation, imputation, diagnostics, evaluation, and pooling, but it does not yet expose all controls available in larger imputation systems. In particular, there is no full predictor-matrix interface, passive-imputation grammar for deterministic derived variables, formal convergence statistic, or automatic tuning layer for learner-backed imputers.

The learner-backed methods should be interpreted as supervised update rules inside a chained stochastic workflow. They can improve predictive recovery, but they do not by themselves guarantee congeniality with a downstream analysis model or valid uncertainty quantification in every setting. For inferential analyses, users should compare methods, inspect trace and distribution diagnostics, evaluate recovery when benchmark truth is available, and assess whether substantive conclusions are robust to plausible imputation choices.

References

- Buuren, Stef van, and Karin Groothuis-Oudshoorn. 2011. "mice: Multivariate Imputation by Chained Equations in R." *Journal of Statistical Software* 45 (3): 1–67. <https://doi.org/10.18637/jss.v045.i03>.
- Chen, Tianqi, and Carlos Guestrin. 2016. "XGBoost: A Scalable Tree Boosting System." In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–94. <https://doi.org/10.1145/2939672.2939785>.
- Friedman, Jerome H. 2001. "Greedy Function Approximation: A Gradient Boosting Machine." *The Annals of Statistics* 29 (5): 1189–1232. <https://doi.org/10.1214/aos/1013203451>.
- Friedman, Jerome, Trevor Hastie, and Robert Tibshirani. 2010. "Regularization Paths for Generalized Linear Models via Coordinate Descent." *Journal of Statistical Software* 33 (1): 1–22. <https://doi.org/10.18637/jss.v033.i01>.
- Josse, Julie, and François Husson. 2016. "missMDA: A Package for Handling Missing Values in Multivariate Data Analysis." *Journal of Statistical Software* 70 (1): 1–31. <https://doi.org/10.18637/jss.v070.i01>.
- Marshall, Andrea, Douglas G. Altman, Roger L. Holder, and Patrick Royston. 2009. "Combining Estimates of Interest in Prognostic Modelling Studies After Multiple Imputation: Current Practice and Guidelines." *BMC Medical Research Methodology* 9: 57. <https://doi.org/10.1186/1471-2288-9-57>.
- Rubin, Donald B. 1987. *Multiple Imputation for Nonresponse in Surveys*. New York: John Wiley & Sons. <https://doi.org/10.1002/9780470316696>.
- Stekhoven, Daniel J., and Peter Buehlmann. 2012. "MissForest: Non-Parametric Missing Value Imputation for Mixed-Type Data." *Bioinformatics* 28 (1): 112–18. <https://doi.org/10.1093/bioinformatics/btr597>.
- Wright, Marvin N., and Andreas Ziegler. 2017. "ranger: A Fast Implementation of Random Forests for High Dimensional Data in C++ and R." *Journal of Statistical Software* 77 (1): 1–17. <https://doi.org/10.18637/jss.v077.i01>.